UNIVERSITY OF ENGINEERING AND TECHNOLOGY LAHORE

Department of Computer Science

Course: Object-Oriented Programming (CS-201) **Session:** 2025-2029 **Semester:** 2nd Semester	**Teacher:** Sir Ali Raza **Submitted:** June 2026 **University:** UET Lahore

INVENTORY MANAGEMENT SYSTEM

A Project Report on Object-Oriented Programming Concepts in C++

PROJECT GROUP MEMBERS

#	Student Name	Roll Number
1	Aman Azam	2025-CS-739
2	Hanzala Bin Imran	2025-CS-748
3	Muhammad Azam	2025-CS-764
4	Muhammad Ramish	2025-CS-769
5	Zain Muhaiuddin	2025-CS-783
6	Muhammad Ahmad	2025-CS-786

Introduction

This project presents a fully functional Inventory Management System developed in C++ that demonstrates the core principles of Object-Oriented Programming (OOP). The system was built as part of the (OOP) course at the University of Engineering and Technology (UET) Lahore by a group of second-semester Computer Science students.

The system manages a warehouse containing multiple inventory sections, each holding various categories of products. It supports full lifecycle management — from adding and removing stock, applying discounts, managing suppliers, to generating invoices and logging transactions.

1. Project Overview

This project implements a console-based Enterprise Inventory Management System in C++. It simulates a large-scale warehouse handling multiple product categories — electronics, groceries, and apparel — each with unique storage, handling, and pricing rules.

The system is built entirely on Object-Oriented Programming principles: a deep inheritance hierarchy, runtime polymorphism, strict encapsulation, composition and aggregation relationships, operator overloading, and a generic template class for audit logging. All data is persisted to a plain-text file between sessions.

2. File Structure

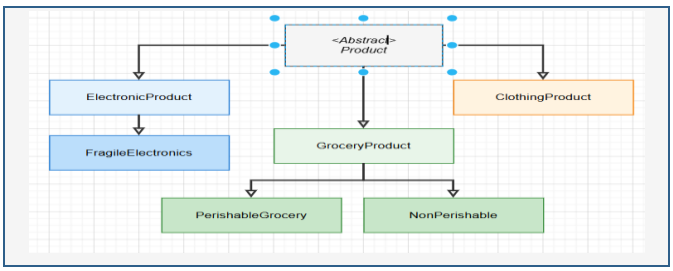
File	Role
product.h	Abstract base class for all products
ElectronicProduct.h	Level-2 electronics base; Rule of Three
FragileElectronics.h	Level-3 specialized electronics
GroceryProduct.h	Level-2 grocery base
PerishableGrocery.h	Level-3 perishable food
NonPerishable.h	Level-3 shelf-stable food
ClothingProduct.h	Level-2 apparel
Supplier.h	Supplier entity; associated with Product
InventorySection.h	Owns Product pointers (Composition)
Warehouse.h	Holds InventorySection pointers (Aggregation)
TransactionLog.h	Generic template audit log
main.cpp	Program entry point and interactive menu
InventorySystem_UML.drawio	UML class diagram

All logic lives in header files. `main.cpp` is the single translation unit, which is the standard approach for single-file C++ lab projects.

3. Class Design and Hierarchy

3.1 Inheritance Hierarchy

Three levels of inheritance are implemented across two branches:



The `Product` class is abstract and cannot be instantiated directly. All concrete subclasses must implement four pure virtual functions: `displayStatus()`, `checkRisk()`, `saveToFile()`, and `clone()`.

3.2 Product (Abstract Base Class)

- `productID` is auto-generated by a private `static int nextID` counter. Each new object gets a unique ID in the format `PROD-001`, `PROD-002`, etc. The counter increments globally because the base constructor is always called first in every subclass chain.
- `price` and `quantity` are validated in the constructor — negative values are clamped to zero.
- `supplier` is a raw pointer to a `Supplier` object. `Product` does not own the supplier; this is an association, not composition.
- `setID()` exists only for file loading, to restore a previously saved ID without triggering the auto-increment.

3.3 ElectronicProduct

Introduces heap-allocated `char* brandName` specifically to demonstrate the Rule of Three. Without the three special members, copying an `ElectronicProduct` would produce two objects sharing the same pointer, causing a double-free on destruction.

Rule of Three:

- **Copy constructor** — allocates new `char[]`, copies each character.
- **Copy-assignment** — deletes old array, allocates new one, copies.
- **Destructor** — `delete[] brandName`.

3.4 FragileElectronics

Level-3 class inheriting from `ElectronicProduct`. Key overrides:

- `applyDiscount()` — caps any discount at 15%.
- `checkRisk()` — flags items with fragility rating above 7.0 as extreme risk.
- `calculateShippingRisk()` — returns `fragilityRating * 0.2` as a risk score (0.0–2.0).

3.5 GroceryProduct

Level-2 base for food. Holds `calories` and `isHalal`. Provides `checkSafety()` for halal confirmation output.

3.6 PerishableGrocery

Level-3 class with `expiryDate` and `storageTemp`. Key behaviour:

- `checkExpiry()` — parses the year from the date string manually and returns true if the year is 2024 or earlier.
- `applyDiscount()` — forces discount to 50% for expired items regardless of input.

- `checkRisk()` — warns if `storageTemp > 8.0 C`.

3.7 NonPerishable

Level-3 class with `shelfLifeYears` and `preservativeLevel`. Flags preservative content above 10% as risky. Provides `getStorageInstructions()`.

3.8 ClothingProduct

Level-2 class with `size`, `fabric`, and `gender`. Caps discounts at 30%. Provides `fitGuide()`.

4. Supporting Classes

4.1 Supplier

Standalone class with `supplierID` and `contractTerms`. Linked to `Product` via a raw pointer — a pure association. `Product` uses `Supplier` but never deletes it. Implements `operator==` by ID and `operator<<` for use in `TransactionLog<Supplier>`.

4.2 InventorySection — Composition

Owns a heap-allocated `Product** products` array. When a section is destroyed, every product in the array is deleted. Deep copying via the copy constructor calls `clone()` on each product, producing a fully independent copy.

Rule of Three:

- **Copy constructor** — calls `clone()` on each product slot.
- **Copy-assignment** — calls `freeAll()` first, then deep copies.
- **Destructor** — `freeAll()` deletes every product then the array.

`operator[]` provides bounds-checked index access. `operator<<` prints a formatted product listing. `getAisleNumber()`, `getCurrentCount()`, and `getCapacity()` are exposed for the UI helper `listSections()`.

4.3 Warehouse — Aggregation

Holds `InventorySection*` pointers with a parallel `bool ownsSection[]` flag array. When `addSection()` is called with `transferOwnership = true`, the Warehouse deletes that section on destruction. With `false`, it only references it.

`merge()` creates a heap-allocated Warehouse, deep-copies all sections using `clone()`, then merges products from the other warehouse by matching IDs. If a match is found, quantities are summed. Otherwise the product is appended. The result Warehouse owns all its sections.

Copy constructor and copy-assignment are **deleted** on `Warehouse` because aggregation ownership semantics make a shallow copy unsafe.

4.4 TransactionLog<T>

Generic template class. Stores up to 200 entries of type `T` in a fixed array. Works with any `T` that supports `operator<<` and copy-assignment.

Instantiated in `main.cpp` with two types:

```
TransactionLog<string>   actionLog;
TransactionLog<Supplier> supplierLog;
```

5. OOP Concepts Used

5.1 Encapsulation

All attributes across all 11 classes are `private` or `protected`. No public data members exist. All setters validate input before assigning. Constructor initializer lists reject negative prices and quantities. Sensitive data such as `supplierID`, `contractTerms`, `price`, and `productID` is never directly accessible from outside the class.

5.2 Deep Inheritance

Three full levels of inheritance are implemented:

- Product → ElectronicProduct → FragileElectronics
- Product → GroceryProduct → PerishableGrocery
- Product → GroceryProduct → NonPerishable

Each level adds its own attributes and overrides virtual functions with behaviour specific to that type.

5.3 Polymorphism

Five virtual functions are overridden across the hierarchy with meaningfully different behaviour per type:

Function	Behaviour
displayStatus()	Each class prints its own specific fields
checkRisk()	Electronics checks voltage; Grocery checks expiry, calories, and temperature; Clothing reports no risk
applyDiscount()	Fragile caps at 15%; Perishable forces 50% if expired; Clothing caps at 30%
saveToFile()	Each class writes exactly its own fields in the correct reload order
clone()	Each class returns new DerivedType(*this) using its own copy constructor

A `Product*` pointer can hold any subclass object, and calling any of these functions dispatches correctly to the actual runtime type.

5.4 Rule of Three

Implemented in both classes that hold heap-allocated pointer members:

ElectronicProduct has `char* brandName`:

- Copy constructor allocates a new array and copies all characters.
- Copy-assignment deletes the old array and allocates a fresh copy.
- Destructor calls `delete[] brandName`.

InventorySection has `Product** products`:

- Copy constructor calls `clone()` on each product, producing independent copies.
- Copy-assignment calls `freeAll()` to release existing products, then deep copies.
- Destructor calls `freeAll()` which deletes every product and then the pointer array.

5.5 Operator Overloading

Operator	Class	Purpose
`operator==`	Product	Two products are equal if their productID matches
`operator==`	Supplier	Two suppliers are equal if their supplierID matches
`operator<<`	Product	Delegates to displayStatus() via virtual dispatch
`operator<<`	Supplier	Prints supplier ID and contract terms
`operator<<`	InventorySection	Prints all products with their slot index
`operator<<`	Warehouse	Full warehouse report including total value
`operator[]`	InventorySection	Bounds-checked access to a product by shelf index

`merge()` on Warehouse implements the conceptual addition — combining two warehouses with quantity-aware product merging.

5.6 Composition and Aggregation

Composition — `InventorySection` fully owns its `Product*` array. Products are allocated outside and transferred in. Removing or destroying a section deletes the products it holds.

Aggregation — `Warehouse` holds section pointers with optional ownership tracked per slot via the `ownsSection[]` flag. Externally created sections are referenced but not deleted by the Warehouse.

5.7 Templates

`TransactionLog<T>` is a fully generic class. It is instantiated with `string` to log action descriptions, and with `Supplier` to log supplier entity objects. Both instances use the same `recordAction()` and `printAuditTrail()` interface, demonstrating true generic reuse.

6. Memory Management

All memory is managed manually with `new` and `delete`. No standard containers or smart pointers are used.

Heap Allocations:

- `ElectronicProduct`: `brandName = new char[...]`
- `InventorySection`: `products = new Product*[capacity]`
- `createProduct()` in `main.cpp`: `new ElectronicProduct(...)`, `new GroceryProduct(...)`, etc.
- `Warehouse::merge()`: `new Warehouse(...)`, `new InventorySection(...)`

Deallocation:

- `~ElectronicProduct()`: `delete[] brandName`
- `InventorySection::freeAll()`: deletes each product, then `delete[] products`
- `~Warehouse()`: deletes only the sections it owns
- The merged Warehouse pointer returned by `merge()` is deleted in `main.cpp` immediately after printing
- Products rejected because a section is full are deleted immediately in `main.cpp` to prevent leaks

7. File Persistence

The system saves and restores inventory state to `inventory_data.txt` on exit and startup.

Each product writes a type-tag as its first token. `loadFromFile` reads the tag and dispatches to the correct constructor. After construction, `setID()` restores the original ID so the auto-increment counter does not overwrite saved IDs on reload.

8. User Interface

The program uses a numbered console menu with options 0 through 8. All input is validated in a loop before acceptance. Integers and doubles are parsed manually without `stoi` or `stod`. Empty strings are rejected. Yes/no prompts accept `y`, `n`, `yes`, `no` in any case.

Before any prompt that requires an aisle label, the `listSections()` helper prints all available sections with their current fill levels (e.g., `[A1 0/20] [B1 0/20]`), preventing the common error of typing a numeric label like `1` when the actual section is named `A1`. On first run, the startup message explicitly names the two default sections created.

9. UML Class Diagram

The diagram shows all 11 classes with their complete attributes and methods. Relationships are represented as follows:

- **Hollow triangle arrow** — Inheritance
- **Filled diamond arrow** — Composition (InventorySection owns Product objects)
- **Hollow diamond arrow** — Aggregation (Warehouse holds InventorySection references)

- **Dashed open arrow** — Association (Product references Supplier) and template use dependencies

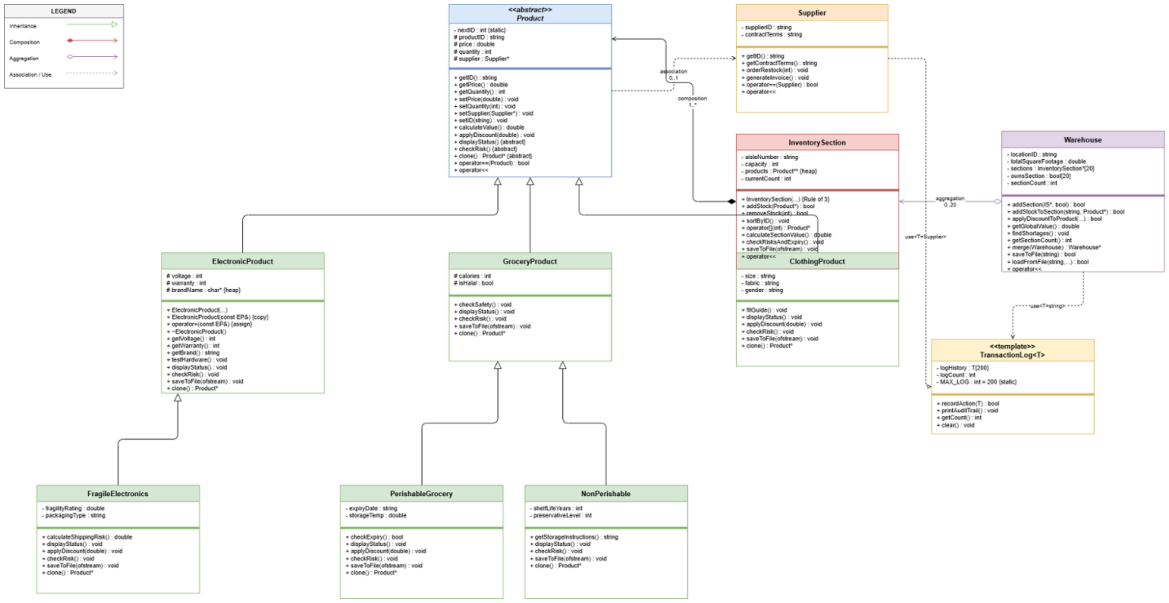


Figure 1 — UML Class Diagram: Enterprise Inventory Management System

Relationship Summary

Relationship	From	To	Type / Multiplicity
Inheritance	`ElectronicProduct`	Product	Generalisation
Inheritance	`GroceryProduct`	Product	Generalisation
Inheritance	`ClothingProduct`	Product	Generalisation
Inheritance	`FragileElectronics`	ElectronicProduct	Generalisation
Inheritance	`PerishableGrocery`	GroceryProduct	Generalisation
Inheritance	`NonPerishable`	GroceryProduct	Generalisation
Association	Product	Supplier	0..1
Composition	InventorySection	Product	1..*
Aggregation	Warehouse	InventorySection	0..20
Dependency	`TransactionLog<T>`	string, Supplier	<T> use

10. Program Output

The following screenshots document the complete execution of the Enterprise Inventory Management System, demonstrating each menu option and the associated program behaviour.

10.1 Main Menu

The console menu is displayed at program startup after the warehouse is initialised. Two default sections, A1 and B1 (capacity 20 each), are created automatically. Options 0 through 8 cover all major operations.

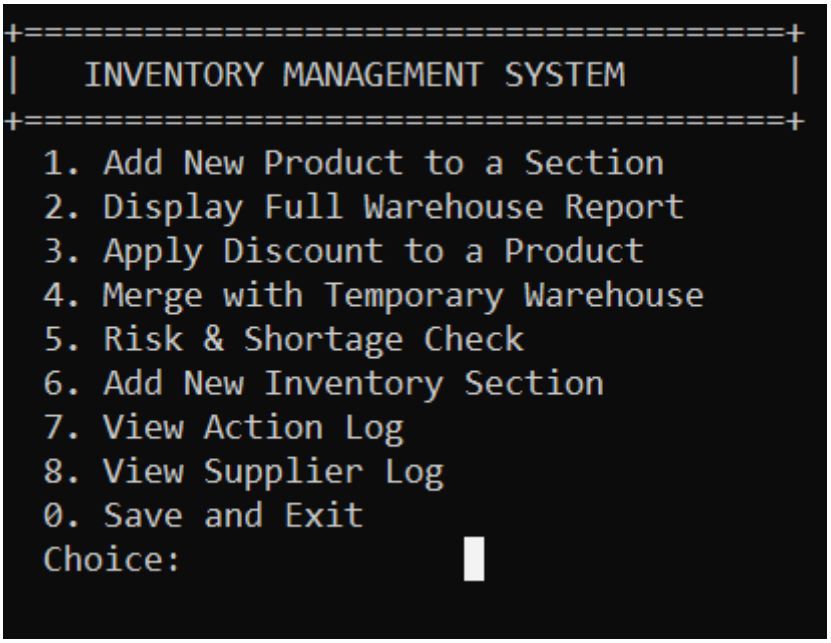


Figure 2 — Main Menu: program start, showing all 9 menu options

10.2 Option 0 — Save and Exit

Selecting option 0 writes all warehouse data to `inventory_data.txt` and terminates the program with a confirmation message: "Inventory saved. Goodbye."

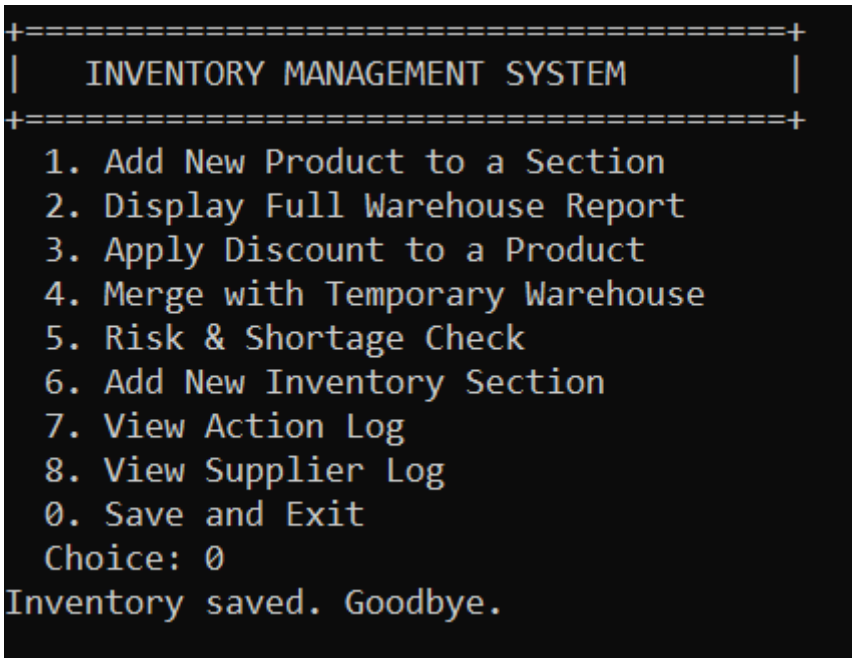


Figure 3 — Option 0: Save and Exit — confirms data written to inventory_data.txt

10.3 Option 1 — Add New Product to a Section

The user selects a target section (e.g., A1), then chooses from 6 product types. The example below adds an LG Electronic Product at $10.00, quantity 2, with 12V voltage and 2-month warranty. The system assigns the auto-generated ID `PROD-001`.

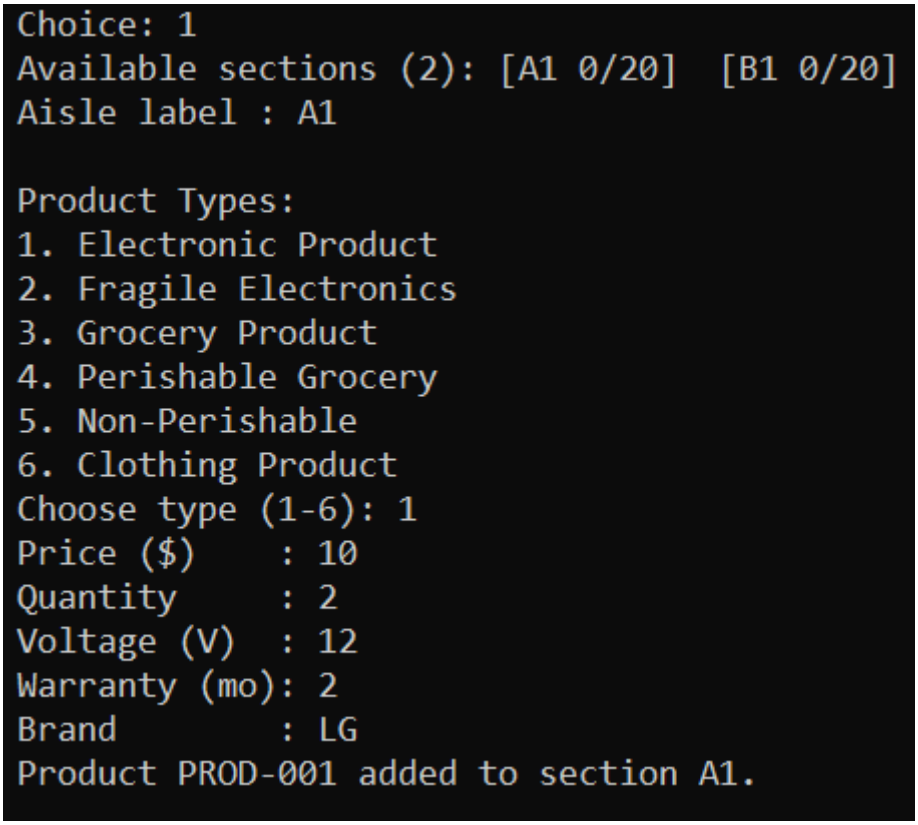


Figure 4 — Option 1: Adding an LG Electronic Product — auto-generated PROD-001 assigned to Section A1

10.4 Option 2 — Display Full Warehouse Report

The full warehouse report shows the location (MAIN-WH-01), area (8,500 sq ft), and section count (2). Each section lists all products with their attributes. Section A1 contains PROD-001; Section B1 is empty. Total inventory value: $20.00.

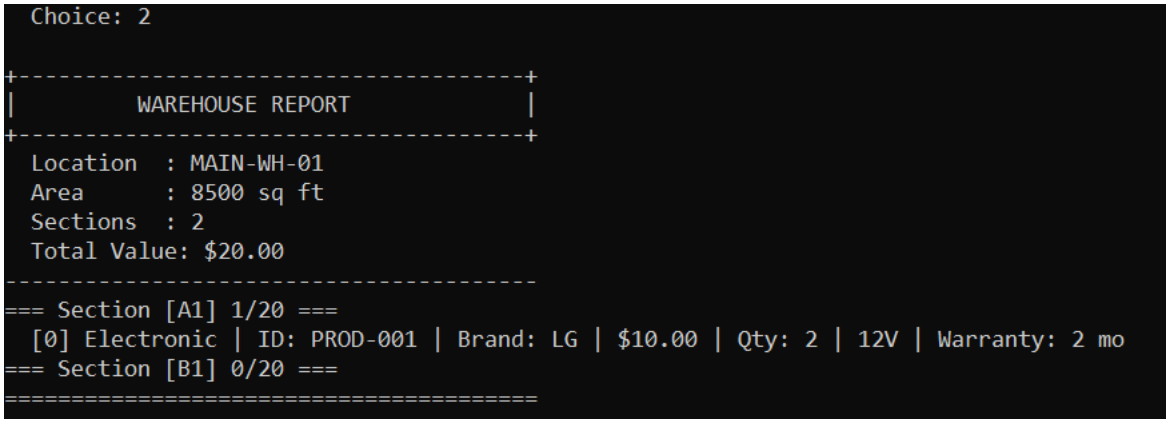


Figure 5 — Option 2: Full Warehouse Report — MAIN-WH-01 with 1 product (PROD-001) and total value $20.00

10.5 Option 3 — Apply Discount to a Product

The user selects the section (A1), the product index (0), and enters a discount percentage (2%). The system applies the discount and displays the confirmation "Discount applied." For `FragileElectronics`, the discount is capped at 15%; for `ClothingProduct`, it is capped at 30%.

```
  Choice: 3

+--------------------------------------+
|         WAREHOUSE REPORT             |
+--------------------------------------+
  Location  : MAIN-WH-01
  Area      : 8500.00 sq ft
  Sections  : 2
  Total Value: $20.00
----------------------------------------
=== Section [A1] 1/20 ===
  [0] Electronic | ID: PROD-001 | Brand: LG | $10.00 | Qty: 2 | 12V | Warranty: 2 mo
=== Section [B1] 0/20 ===
========================================
  Available sections (2): [A1 1/20]  [B1 0/20]
  Aisle label     : A1
  Product index   : 0
  Discount (0-100): 2
  Discount applied.
```

Figure 6 — Option 3: Applying a 2% discount to PROD-001 in Section A1

10.6 Option 4 — Merge with Temporary Warehouse

A temporary warehouse (TEMP-WH) is created with a Sony Fragile Electronics product (PROD-002: $30.00, qty 21, fragility 2/10). The merged result warehouse is labelled MAIN-WH-01_+_TEMP-WH with a combined area of 9,500 sq ft and a total inventory value of $649.60. Section A1 now holds both products; Section B1 remains empty.

```
  Choice: 4

  Creating a temporary warehouse for merge...
  Add a product to the temporary warehouse:

  Product Types:
  1. Electronic Product
  2. Fragile Electronics
  3. Grocery Product
  4. Perishable Grocery
  5. Non-Perishable
  6. Clothing Product
  Choose type (1-6): 2
  Price ($)     : 30
  Quantity      : 21
  Voltage (V)   : 12
  Warranty (mo): 21
  Brand         : Sony
  Fragility (1-10): 2
  Packaging type  : Default

  Merged Warehouse:

+--------------------------------------+
|         WAREHOUSE REPORT             |
+--------------------------------------+
  Location   : MAIN-WH-01_+_TEMP-WH
  Area       : 9500.00 sq ft
  Sections   : 2
  Total Value: $649.60
----------------------------------------
=== Section [A1] 2/40 ===
  [0] Electronic | ID: PROD-001 | Brand: LG | $9.80 | Qty: 2 | 12V | Warranty: 2 mo
  [1] Electronic | ID: PROD-002 | Brand: Sony | $30.00 | Qty: 21 | 12V | Warranty: 21 mo
           Fragile | Rating: 2.00/10 | Packaging: Default
=== Section [B1] 0/40 ===
========================================
```

Figure 7 — Option 4: Merging with TEMP-WH — resulting combined warehouse report

10.7 Option 5 — Risk and Shortage Check

The risk check for PROD-001 (voltage within safe range) returns [OK]. Both Section A1 (1 item) and Section B1 (0 items) trigger [SHORTAGE] alerts. The report also evaluates expiry dates, storage temperatures, and preservative levels depending on product type.

```
  Choice: 5

--- Risk & Shortage Report ---
[OK]   Voltage safe - PROD-001
[SHORTAGE] Section A1 has only 1 item(s).
[SHORTAGE] Section B1 has only 0 item(s).
```

Figure 8 — Option 5: Risk and Shortage Report — voltage OK, shortage flags on A1 and B1

10.8 Option 6 — Add New Inventory Section

The user enters a new aisle label (C1) and a capacity (30). The section is created and added to the warehouse. The confirmation message "Section 'C1' added." is displayed.

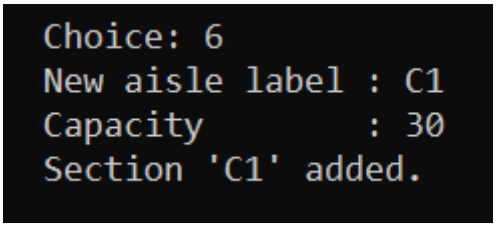


Figure 9 — Option 6: Adding new Section C1 with capacity 30

10.9 Option 7 — View Action Audit Log

The `TransactionLog<string>` audit trail prints all recorded string entries in order:

- Initialized warehouse with default sections A1 and B1
- Added PROD-001 to aisle A1
- Discount 2% applied in A1[0]
- Merged with TEMP-WH
- Added section C1

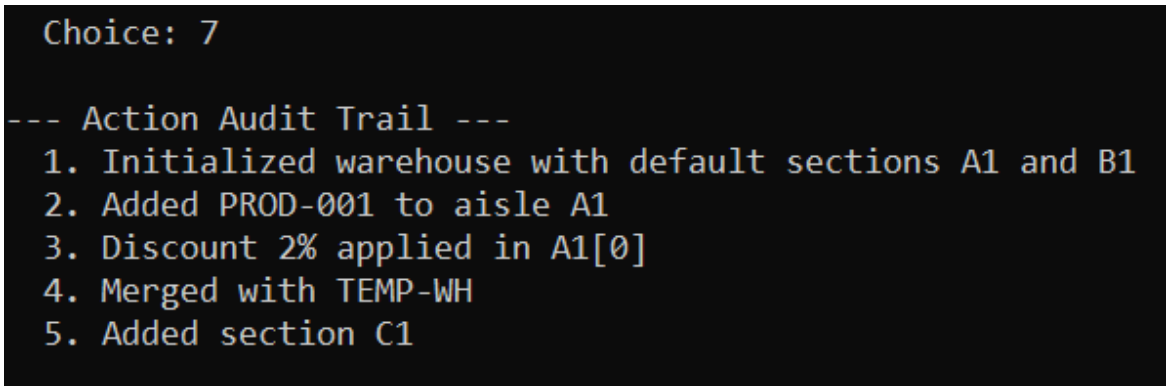


Figure 10 — Option 7: Action Audit Log — 5 recorded string entries via TransactionLog<string>

10.10 Option 8 — View Supplier Audit Log

The `TransactionLog<Supplier>` prints all logged Supplier objects. Three suppliers are recorded:

- SUP-001 — Net 30 days
- SUP-002 — Prepaid, 10% discount
- SUP-003 — Net 15, perishables only

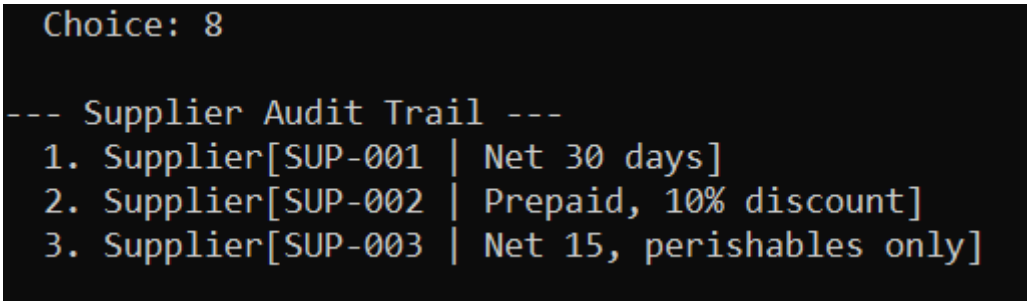


Figure 11 — Option 8: Supplier Audit Log — 3 Supplier objects logged via TransactionLog<Supplier>

— End of Report —